

## Software Engineering, Lab 3

### Component Modelling and Architectural Pattern Selection

Name: Abhigyan Dutta

SRN: PES2UG24CS019

Class: 5A, CSE

Date: 13 September 2026

Problem Statement 17: Vaccination Cohort and Dose Scheduling System

Domain: Healthcare and Telemedicine

**We chose Microservices Architecture for the Vaccination Cohort and Dose Scheduling System.**

#### Two reasons

- The system already breaks into five distinct functional areas, registration and cohort assignment, booking and interval enforcement, dose recording, certificate issuance and verification, and policy administration, and these were run as five separate epics in Lab 2 with their own sprints. Microservices lets each of these stay a separate deployable service with its own codebase and release cycle, instead of folding five independent concerns back into one shared application.
- The system already depends on two services it does not control, the National ID Service for identity checks and the Certificate Authority for signing. Microservices gives each of these its own service boundary with its own retry and failure handling, so a slow or unavailable external call does not stall the rest of the system the way it could inside a single monolithic process.

#### Security advantage

Only the Certificate Service holds the signing and verification logic and the credentials needed to talk to the Certificate Authority. If any other service, such as Booking, were compromised, the attacker still could not sign or forge a certificate, because that capability never leaves the one service that needs it. Keeping the boundary this way limits the blast radius of a breach instead of trusting every service equally.

#### Performance benefit

Booking is the component under the heaviest load, since the requirement is 5000 concurrent bookings at p95 under 2 seconds during the daily rush when a cohort opens. Microservices lets the Booking Service be scaled out on its own during that window, while a rarely used component like Policy Configuration stays at a single instance. A layered or client-server design would scale the whole application together, paying for capacity that most of the system does not need at that moment.